Queries executed on the alias will now fetch data from the new index without the application being bounced. Thus, we've achieved zero downtime.

MULTIPLE ALIASING OPERATIONS USING THE _ALIASES API

In addition to working with aliases using the `_alias` API, we can use another API to work on multiple aliasing actions: the `_aliases` API. It combines several actions, such as adding and removing aliases and deleting the indexes. Whereas the `_alias` API is used to create an alias, the `_aliases` API helps create multiple actions on indexes related to aliasing.

The code in listing 6.19 performs two distinct aliasing operations on two indexes: it removes an alias pointing to an old index and repoints (via the `add` action) the same alias to a new index (see the previous section to understand the need for doing this).

Listing 6.19 Performing multiple aliasing operations

```
POST _aliases
{
  "actions": [
    {
      "remove": {
        "index": "vintage_cars",
        "alias": "vintage_cars_alias"
      }
    },
    {
      "add": {
        "index": "vintage_cars_new",
        "alias": "vintage_cars_alias"
      }
    }
  ]
}
```

Uses the _aliases API to execute multiple actions

Lists the individual actions

Removes the alias that points to an old index

Adds an alias that points to a new index

We remove an alias, `vintage_cars_alias`, that was originally created for the `vintage_cars` index. We then reassign it to the `vintage_cars_new` index.

With the `_aliases` API, we can also delete the alias pointing to the existing index and assign it to a new index when the new index is ready after migrating the data. And we can create an alias for multiple indexes using the same `_aliases` API by using the `indexes` parameter to set up the list of indexes. The following listing shows how to do that.

Listing 6.20 Creating an alias pointing to multiple indexes

```
POST _aliases
{
  "actions": [
    {
      "add": {
```